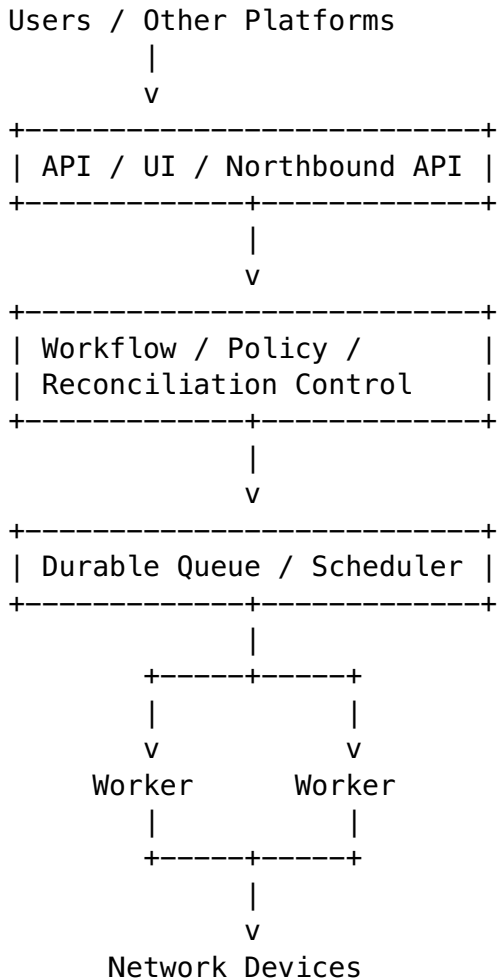


DAY 4 — Kubernetes, Cloud, Distributed Platforms, CI/CD and Data

The mental model for today

A production network-automation system is much more than an API that sends commands to routers.

A mature platform normally has several layers:



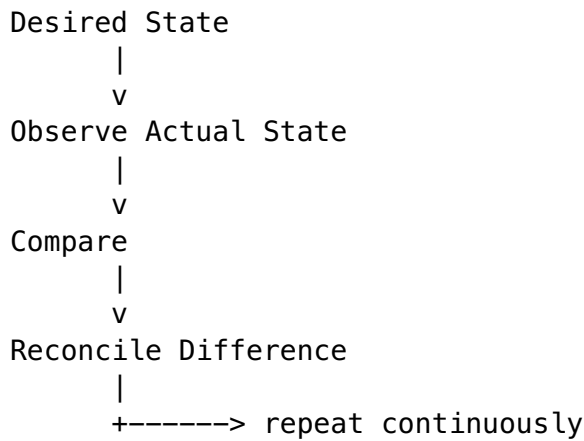
Supporting everything:

Kubernetes	Databases	Cache	Object Storage
Secrets	Monitoring	CI/CD	IaC
Telemetry			

The key architectural lesson is:

Kubernetes automates the lifecycle of software infrastructure in much the same way that the network-automation platform from Day 3 automates the lifecycle of network configuration.

Both depend heavily on:

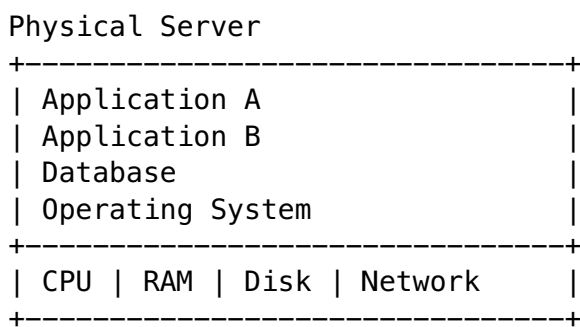


That reconciliation idea is one of the most important concepts connecting Days 3 and 4.

1. Runtime evolution

Bare metal

Originally, applications ran directly on physical servers.



The application shared the machine's operating system and resources.

Advantages

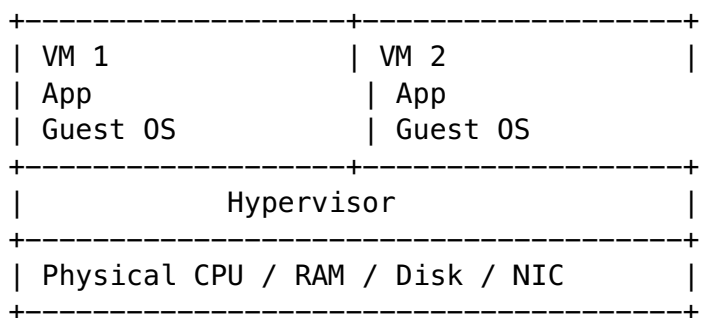
- maximum hardware access
- little virtualization overhead
- useful for specialised/high-performance systems

Problems

- poor isolation
 - difficult provisioning
 - low hardware utilisation
 - one application can affect another
 - replacing or upgrading servers is operationally expensive
-

Virtual machines

Virtualisation added a **hypervisor** between hardware and operating systems.



Hypervisor

Software that divides physical hardware into virtual machines.

Examples conceptually include:

- type-1/bare-metal hypervisors
- host-based hypervisors

Each VM behaves roughly like its own machine.

VM

A VM contains:

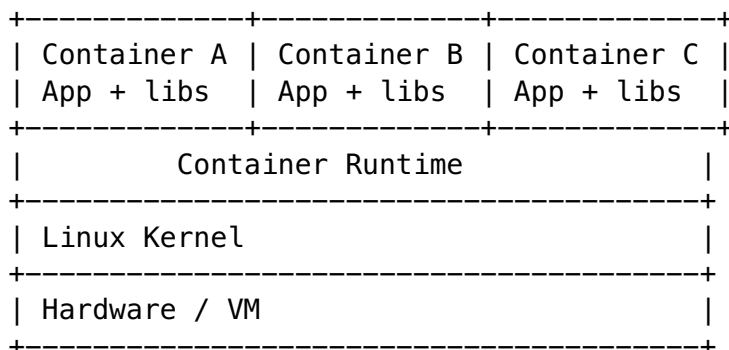
Application
 Libraries
 Guest Operating System
 Virtual CPU/RAM/Disk/NIC

This provides strong isolation.

But every VM normally carries its own OS, making VMs relatively heavy.

Containers

Containers usually share the host operating-system kernel.



Container

A container is an isolated process environment.

It packages:

- application
- libraries
- runtime dependencies
- filesystem content
- startup instructions

but generally **not an entire guest OS**.

Image

An image is the immutable package used to create containers.

Think:

Image = blueprint
Container = running instance of blueprint

For example:

network-worker:v4.2.1

could contain:

Python runtime
Network libraries
Worker application
Certificates
Dependency packages

Container runtime

The runtime actually creates and manages containers.

Conceptually:

Kubernetes
|
v
Container Runtime
|
v
Linux container/process

VM vs container

Characteristic	VM	Container
Isolation	Strong	Process-level
Guest OS	Usually yes	Usually no
Startup	Slower	Faster
Size	GBs common	MBs–GBs

Characteristic	VM	Container
Density	Lower	Higher
Portability	Good	Very good
OS overhead	Higher	Lower
Operational model	Machine-oriented	Application-oriented

Containers did not eliminate VMs.

A common architecture is:

```

Physical Server
  |
Hypervisor
  |
Virtual Machines
  |
Kubernetes
  |
Containers

```

Public-cloud Kubernetes frequently follows exactly this model.

2. Kubernetes architecture

Kubernetes is essentially a distributed control system for running applications.

It separates:

```

Control Plane
  |
  | decides what should happen
  v
Worker Nodes
  |
  | run actual applications
  v
Containers

```

Control plane

API Server

The API server is the front door to Kubernetes.

Clients interact with Kubernetes through it.

kubectl
CI/CD
Operators
Controllers
Applications
|
v
Kubernetes API Server

Requests such as:

Create Deployment
Scale application
Change configuration
Create Service

go through the API server.

etcd

etcd stores Kubernetes cluster state.

For example:

Deployment requires 5 replicas
Service X exists
Secret Y exists
Node A exists
Pod B is assigned to Node C

Conceptually:

API Server
|
v
etcd

It is therefore critical infrastructure.

Scheduler

The scheduler determines **which worker node should run a new Pod**.

Suppose:

worker-1: 70% allocated
worker-2: 40% allocated
worker-3: insufficient memory

A new Pod requiring:

CPU: 1
RAM: 2 GB

might be assigned to worker-2.

Scheduling can consider:

- resource requirements
- node constraints
- affinity
- anti-affinity
- topology
- taints/tolerations
- resource availability

Controller Manager

Controllers continuously compare:

Desired state
vs
Observed state

For example:

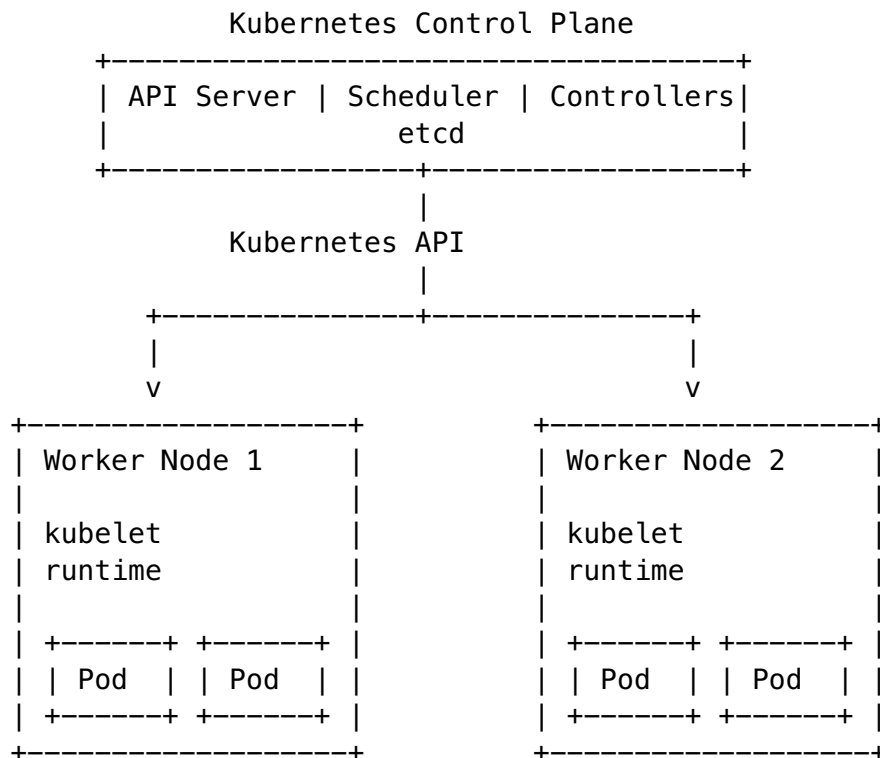
Desired replicas = 5
Actual replicas = 3

Controller action:

Create 2 more Pods

This is reconciliation.

Worker architecture



Node

A machine participating in the Kubernetes cluster.

It can be:

- physical server
- VM
- cloud instance

Kubelet

The kubelet runs on each worker.

Its responsibility is approximately:

| Make sure Pods assigned to this node are actually running.

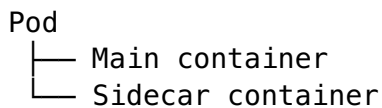
Pod

A Pod is Kubernetes' basic deployable unit.

Usually:

Pod
└─ Application container

Sometimes:



Containers inside the same Pod share important resources such as networking.

3. Kubernetes declarative model

This is one of today's most important concepts.

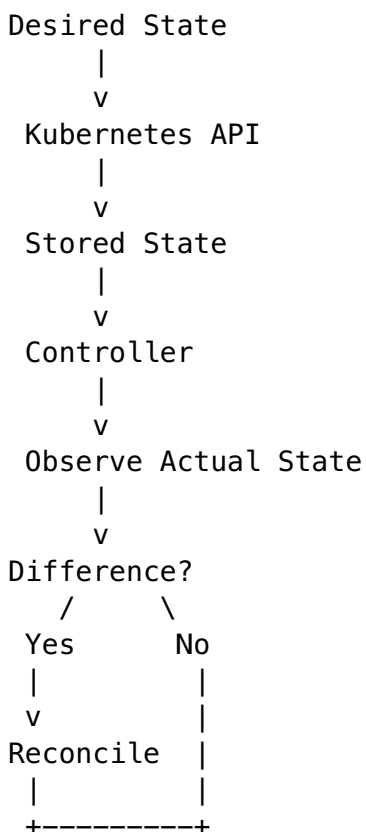
You don't normally say:

Start process X.
Then start another.
Then check it.
Then restart if broken.

Instead you declare:

I want 5 instances of application X.

Kubernetes continually attempts to maintain that state.



Connection to Day 3 network automation

The architecture is nearly identical.

Kubernetes

Desired:
5 application replicas

Observed:
3 replicas

Action:
Create 2 replicas

Network automation

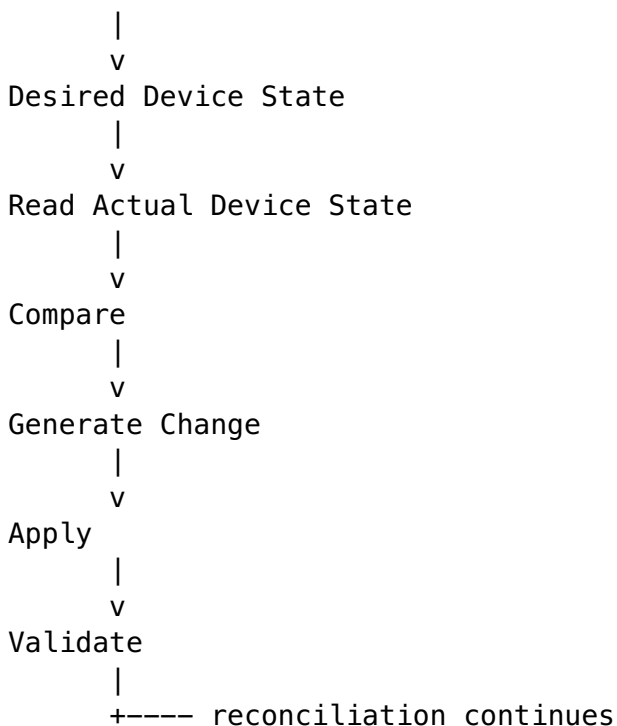
Desired:
BGP neighbour configured

Observed:
BGP neighbour missing

Action:
Generate/apply configuration

Or:

Source of Truth



This architecture is fundamentally more powerful than imperative scripting.

4. Kubernetes workloads

Pod

Smallest runtime unit.

You normally shouldn't manage large numbers of Pods manually.

Higher-level controllers manage them.

ReplicaSet

Ensures a specified number of equivalent Pods exist.

Desired = 4

Pod
Pod
Pod

ReplicaSet notices only 3

|
v

Creates another Pod

Deployments generally manage ReplicaSets for you.

Deployment

Designed primarily for stateless applications.

It provides:

- replicas
- rolling updates
- rollback
- replacement of failed Pods

Ideal for:

Northbound API

UI/backend

Network worker fleet

Policy service

Example:

Network API Deployment

|
+-- Pod
+-- Pod
+-- Pod

StatefulSet

Used where workload identity or storage stability matters.

Typical properties include:

- stable Pod identity
- ordered lifecycle
- stable persistent storage association

Suitable for certain:

databases
message brokers
distributed stateful systems

Although in cloud environments, managed databases/message brokers are often preferable.

DaemonSet

Runs a Pod on every relevant node.

Node A --> Collector
Node B --> Collector
Node C --> Collector
Node D --> Collector

Excellent for:

- monitoring agents
- log collectors
- node-level networking agents
- security agents

A telemetry collector that must observe every Kubernetes node might use a DaemonSet.

Job

Runs work until completion.

Example:

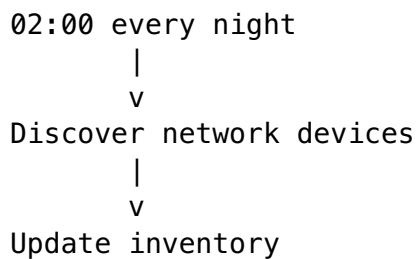
Import 100,000 network inventory records

Once the task finishes, the Job completes.

CronJob

Creates Jobs according to a schedule.

For example:



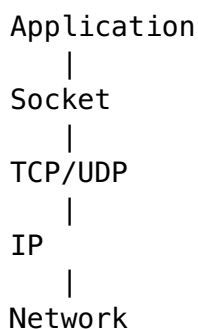
Mapping to our platform

Platform capability	Kubernetes workload
API service	Deployment
Network worker	Deployment
Web UI	Deployment
Telemetry agent per node	DaemonSet
One-off database migration	Job
Scheduled device discovery	CronJob
Certain databases/brokers	StatefulSet
Policy service	Deployment

5. Kubernetes networking

This connects directly to Days 1 and 2.

Remember:



Kubernetes doesn't remove networking fundamentals.

It builds abstractions on top of them.

Pod networking

Each Pod normally receives its own IP address.

Pod A
10.20.1.5

Pod B
10.20.2.9

Conceptually:

Pod A
|
Pod Network
|
Pod B

The underlying implementation varies.

CNI

CNI stands for **Container Network Interface**.

It defines how networking plugins integrate container networking with Kubernetes.

A CNI implementation may handle:

- Pod interfaces
- IP allocation
- routing
- encapsulation
- network policy enforcement

Think:

Kubernetes says:
"this Pod needs networking"

|
v

CNI implementation establishes
the required connectivity

Service

Pods are ephemeral.

Suppose API Pods are:

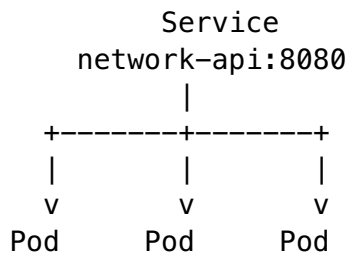
10.1.1.4
10.1.2.8
10.1.3.7

After restarts:

10.1.2.17
10.1.3.24
10.1.4.11

Clients should not track individual Pod addresses.

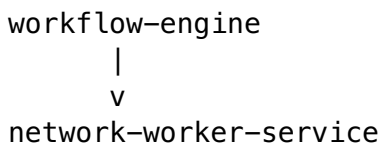
A Kubernetes **Service** provides a stable service endpoint.



ClusterIP

Makes the Service available primarily within the cluster.

Example:

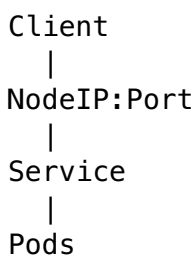


No public access is required.

NodePort

Exposes a port through Kubernetes nodes.

Conceptually:



It is useful conceptually and for some specialised deployments, although production application exposure often uses other mechanisms.

LoadBalancer

Requests/exposes a load-balanced endpoint through supporting infrastructure.

Conceptually:

```
Worker
|
DNS lookup
|
workflow-service
|
Service IP
|
Workflow Pods
```

NetworkPolicy

NetworkPolicy restricts communication between workloads where supported by the cluster's networking implementation.

Without segmentation:

```
Pod A ----- Pod B
 \           /
  \         /
   \       /
    \     /
     \   /
      \ /
       Pod C
```

With appropriate policy:

```
API ----> Workflow
          |
          v
        Queue
          |
          v
        Workers
```

Unrelated Pod ----X----> Database

This is essentially micro-segmentation for Kubernetes workloads.

6. Kubernetes controllers and operators

Controller pattern

A controller runs:

```
Observe
|
Compare
|
Reconcile
|
Observe again
```

For example:

Deployment says replicas=4

Actual Pods=3

Controller:
create one Pod

Custom Resource Definition — CRD

Kubernetes has built-in resources such as:

Pod
Deployment
Service
Job

A CRD lets us introduce domain-specific resources.

Imagine:

NetworkSite

Desired state:

- Site: Bangalore-DC1
- Policy: Gold
- DiscoveryEnabled: true
- WorkerRegion: India

Kubernetes can now represent this object through its API.

Operator

An Operator is a controller containing domain-specific lifecycle knowledge.

For example, a database operator could understand:

Provision cluster
Initialize members
Perform backups
Upgrade safely
Replace failed members
Restore cluster

So:

CRD
+
Controller
+
Domain lifecycle logic
=
Operator

Operators are powerful when lifecycle management involves more than simply "start this container."

7. Configuration and secrets

ConfigMap

Used for non-secret application configuration.

Examples:

```
LOG_LEVEL=INFO  
DISCOVERY_BATCH_SIZE=100  
WORKER_REGION=india-south
```

Kubernetes Secret

Used to represent sensitive configuration.

Examples:

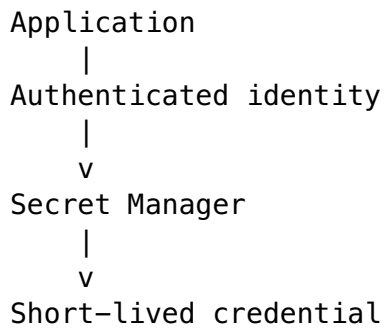
```
database password  
API token  
TLS private material  
device credential reference
```

A Kubernetes Secret should still be treated as sensitive data. It is not equivalent to a full enterprise secret-management strategy.

External secret manager

Large platforms often use a dedicated secret service.

Architecture:

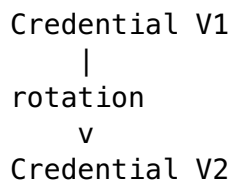


rather than embedding credentials into:

Git
container image
application source
pipeline variables

Credential rotation

Credentials should be replaceable without rebuilding applications.



Applications should ideally refresh credentials safely.

Why network platforms have extreme secret requirements

A network worker may have permission to modify:

routers
switches
firewalls
load balancers
cloud networks

Compromise of one credential could therefore have broad blast radius.

Prefer:

Short-lived credentials
+
Least privilege
+
Central secret manager
+
Audit
+
Rotation
+
Environment separation

8. Resource management and scheduling

Kubernetes needs to understand application resource requirements.

CPU request

The CPU capacity Kubernetes should plan for when scheduling a Pod.

```
request = 500 millicores
```

means roughly half a CPU core as scheduling demand.

CPU limit

Maximum CPU allocation configured for the container.

When demand exceeds available permitted CPU, execution can be throttled.

Memory request

Memory capacity Kubernetes reserves/plans during scheduling.

Memory limit

Maximum memory the container may consume.

Exceeding memory limits may result in the workload being terminated for out-of-memory conditions.

Why requests matter

Imagine:

Node capacity = 16 GB

Worker A request = 4 GB

Worker B request = 4 GB

Worker C request = 4 GB

Worker D request = 4 GB

Kubernetes knows the node is effectively full for scheduling purposes.

Without realistic requests, workloads may be densely packed and fail under load.

Resource pressure

Nodes can experience:

CPU pressure

memory pressure

disk pressure

Under severe resource pressure, Kubernetes may need to evict certain workloads.

HPA — Horizontal Pod Autoscaler

HPA changes replica count based on metrics.

Example:

Worker Pods = 5

Queue depth rises

CPU rises

|
v

HPA

|
v

Worker Pods = 15

CPU is one possible signal. Mature systems can use workload-specific signals such as queue depth when appropriate.

9. Health and lifecycle

Containers being "running" does not necessarily mean they are useful.

Startup probe

Answers:

| Has the application finished starting?

Useful for applications that take significant time to initialise.

Readiness probe

Answers:

| Can this instance accept traffic right now?

Example:

```
Pod starts
  |
loads configuration
  |
connects database
  |
initialises dependencies
  |
ready=true
```

Traffic should only then reach the Pod.

Liveness probe

Answers:

| Is this process still functioning sufficiently to remain running?

If repeatedly unhealthy, Kubernetes can restart it.

Important distinction

Startup -> Did startup complete?

Readiness -> Should traffic reach it?

Liveness -> Should Kubernetes restart it?

Poor liveness-probe design can cause restart loops, so probes must test meaningful but appropriate health conditions.

Graceful shutdown

Suppose a network worker is configuring a router.

Bad behaviour:

Kubernetes terminates Pod

|
v

Process immediately dies

|
v

Change operation interrupted

Better:

Termination signal

|

Stop accepting new jobs

|

Finish/checkpoint current work

|

Release locks

|

Update job state

|

Exit

Rolling restart/update

Instances are replaced incrementally.

V1 V1 V1 V1

V2 V1 V1 V1

V2 V2 V1 V1

V2 V2 V2 V1

V2 V2 V2 V2

This avoids taking the whole service offline.

Pod disruption

Pods can disappear because of:

- maintenance
- node failure
- cluster upgrades
- scaling
- infrastructure failures

Applications must therefore assume:

| Any individual Pod can disappear.

10. Persistent storage

Stateless workload

Does not require local persistent identity/state to process future requests.

Example:

```
API Pod
  |
read/write
  |
Database
```

If the API Pod disappears:

```
New API Pod
  |
same database
  |
continue
```

Good candidates:

- API services
 - policy evaluation services
 - many worker processes
 - UI/backend services
-

Stateful workload

Maintains information that must survive process replacement.

Examples:

```
database
message broker
persistent search index
```

Persistent Volume concept

Provides storage whose lifetime is decoupled from an individual container.

```
Pod
  |
Persistent Volume
  |
Storage backend
```

Pod disappears:

New Pod
|
same durable volume

StorageClass

Represents a class/type of storage available to workloads.

For example conceptually:

fast SSD
standard disk
high durability storage

The application requests a storage requirement rather than manually provisioning disks.

What should be stateless in our platform?

Prefer statelessness for:

API
UI backend
policy evaluator
worker fleet
authentication proxy
transformation services

Durable data should instead live in systems designed for state.

Persist:

Inventory
Workflow state
Audit history
Job status
Source-of-truth data
Telemetry
Configuration snapshots

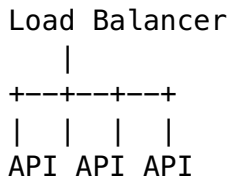
11. High availability

HA means the system continues providing acceptable service despite failures.

It is not simply:

| run two copies.

API

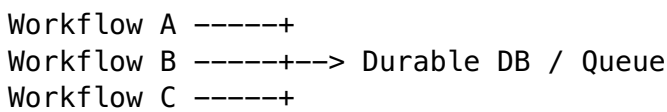


Any individual API Pod can fail.

Workflow engine

Run multiple instances.

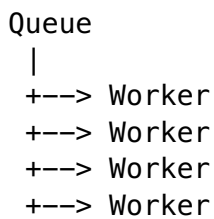
Persistent workflow state must not exist only in a Pod's memory.



Some scheduler functions may require coordination or leader election.

Workers

Worker fleets naturally scale horizontally.

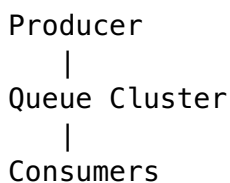


Jobs must be:

- idempotent where possible
 - retry-aware
 - durably acknowledged
-

Queue

A queue generally needs replicated durable storage.



If one broker fails, another should preserve service.

Database

Database HA typically involves:

Primary
|
replication
|
Replica(s)

Failover strategy depends on database architecture and consistency requirements.

Cache

Cache failure should ideally degrade performance rather than destroy correctness.

Cache available:
2 ms

Cache unavailable:
DB lookup = 30 ms

The platform should usually remain correct.

Kubernetes control plane

Production clusters typically have multiple control-plane members so one machine failure does not destroy cluster management.

Failure domains

A failure domain is a boundary within which failures may be correlated.

Examples:

server
rack
data centre
availability zone
region
cloud provider

Putting two replicas on the same physical host provides less resilience than distributing them.

Availability zones

Conceptually:

```
Region
|
+--- AZ-A
|   +--- API
|   +--- Worker
|
+--- AZ-B
|   +--- API
|   +--- Worker
|
+--- AZ-C
    +--- API
    +--- Worker
```

Now one AZ failure does not necessarily eliminate the application.

12. Distributed-system foundations

These concepts become critical once our platform spans many processes and machines.

Stateless vs stateful

Stateless API:

```
Request
|
Any API Pod
|
Database
```

Easy to scale.

Stateful application:

Request requires local session/state

Harder to move and recover.

Therefore:

| Push durable state into specialised durable systems where practical.

Horizontal scaling

Instead of:

```
1 worker
8 CPUs
```

we can run:

8 workers
1 CPU each

Especially useful for independent network jobs.

Replication

Maintain multiple copies.

Examples:

database replicas
queue replicas
API replicas
configuration snapshot copies

Purpose:

- availability
 - read scaling
 - disaster resilience
-

Partitioning

Divide a large dataset/workload.

Example:

Devices 1–1M

partition by region:

India
Europe
Americas
APAC

Workers could consume regional partitions.

Partitioning increases scalability but introduces coordination complexity.

Strong consistency

After a successful write, subsequent reads should observe the required latest consistent state according to the datastore's consistency model.

Useful for operations such as:

workflow ownership
financial/security-sensitive state
lock state
critical policy transition

Eventual consistency

Copies may temporarily disagree but converge later.

This can be perfectly acceptable for:

search indexes
analytics views
telemetry dashboards
inventory summary caches

if the business semantics permit it.

Durable messaging

A network job must not disappear because one worker crashes.

API
|
Durable Queue
|
Worker receives Job
|
Worker crashes
|
Job becomes available again
|
Other Worker

This is fundamentally different from an in-memory Python list.

Distributed locking

Suppose two workers attempt:

Configure Router-27

at the same time.

Possible solution:

Acquire logical device lock
|
Worker A succeeds
Worker B waits/rejects

But distributed locks add complexity:

- expiration
- crashes
- stale owners
- fencing
- clock/time assumptions

Often good job partitioning/idempotency can reduce lock dependency.

Leader election

Certain operations should run only once.

Example:

Scheduler A
Scheduler B
Scheduler C

All are alive, but one becomes leader:

Leader:
runs global reconciliation scan

Followers:
stand by

When the leader fails, another takes over.

Failure isolation

One subsystem's failure should not collapse everything else.

Example:

Telemetry pipeline overloaded
|
X
|
Network configuration API remains healthy

Techniques include:

- queues
- bounded concurrency
- separate worker pools
- timeouts
- rate limits
- circuit breaking

- separate resource quotas
-

13. Data-store choices

There is no universally best database.

Choose according to access patterns and consistency needs.

Relational database

Excellent where entities and relationships matter.

Examples:

devices
sites
workflow records
users
approvals
audit metadata
change requests

Benefits:

- transactions
 - constraints
 - joins
 - mature querying
-

NoSQL / key-value

Useful when:

- access pattern is simple
- scale is very high
- schema is flexible
- key-based retrieval dominates

Example:

device_id -> transient calculated state

Do not choose NoSQL merely because the platform is large.

Cache

For temporary/frequently reused values.

Examples:

recent device capability information
reference data
short-lived discovery result
expensive computed summary

The authoritative value should usually live elsewhere.

Object storage

Excellent for large immutable objects.

Examples:

configuration snapshots
device backups
firmware metadata
export files
telemetry archives
large reports

Typical pattern:

Database:
snapshot_id, timestamp, device_id, object_uri

Object store:
actual large snapshot file

Search/index platform

Useful for:

logs
audit search
full-text configuration search
telemetry investigation

Its strength is indexing/querying rather than transactional system-of-record behaviour.

14. Database engineering

Transactions

A transaction groups changes into a logical unit.

Example:

Create workflow
Create workflow tasks
Create audit event

We may require:

all succeed
or
none succeed

Isolation

Concurrent transactions can interact in unexpected ways.

Isolation defines how much one transaction can observe effects from another.

The trade-off is often:

More isolation
|
more predictable behaviour
|
potentially higher coordination/cost

Choose based on business correctness, not maximum isolation everywhere.

Indexes

Indexes accelerate selected queries.

Without index:

Search 20M records
|
scan many rows

With appropriate index:

device_id lookup
|
index
|
small targeted lookup

But indexes cost:

- storage
 - write overhead
 - maintenance
-

Connection pools

Opening a new DB connection for every request is expensive.

Instead:

```
Application
  |
Connection Pool
  | | | | |
DB connections
```

Applications borrow and return connections.

Pools must be bounded. Otherwise hundreds of Pods can overwhelm a database.

Optimistic locking

Useful when conflicting updates are uncommon.

Imagine:

```
workflow version = 7
```

Worker tries:

```
update workflow
where version = 7
```

If another process already changed it to version 8, the update fails instead of silently overwriting newer state.

Schema migrations

Production schema changes must be version-controlled.

Example:

```
V1 devices table
  |
Migration
  v
V2 adds lifecycle_status
```

Applications and schema changes should support safe deployment ordering.

Query scalability

Be careful with:

```
SELECT everything
large unbounded joins
N+1 queries
full-table scans
```

Use:

- indexes
 - bounded result sets
 - appropriate data models
 - partitioning where justified
 - query-plan inspection
 - read replicas where appropriate
-

Pagination

Never return a million devices in one HTTP response.

Prefer:

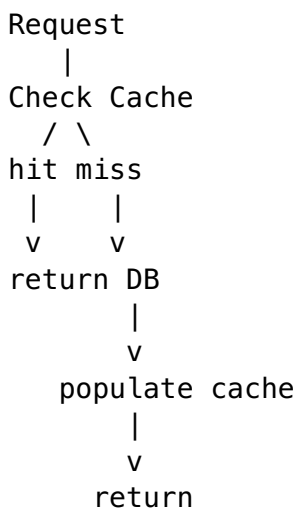
GET devices?limit=100

For large/changing datasets, cursor-based pagination is often more predictable than naive offsets.

15. Caching

Cache-aside

Application owns cache population.



TTL

TTL means time to live.

inventory-summary cached for 60 sec

After that it expires.

Cache invalidation

Suppose:

DB device status = ACTIVE
Cache device status = ACTIVE

Then database changes:

DB = RETIRED

Cache must eventually stop returning ACTIVE.

This can happen through:

- expiration
 - explicit invalidation
 - event-driven update
-

Stale data

Caches inherently create the possibility that clients see older information.

Ask:

| Is temporary staleness acceptable for this data?

Cache stampede

Suppose 10,000 requests use one cached object.

It expires.

10,000 callers
|
cache miss
|
10,000 database requests

This is a cache stampede.

Mitigations include:

- jittered expirations
 - request coalescing
 - background refresh
 - logical locking
-

What should be cached?

Good candidates:

expensive read-only lookup
device metadata summaries
UI summary statistics
slow external API responses

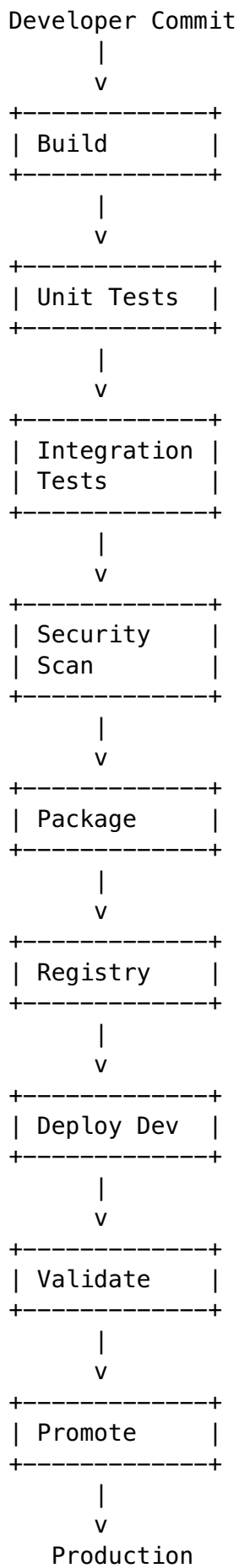
Be careful caching:

current distributed-lock ownership
authoritative workflow state
critical security decisions
fresh device configuration state
data where stale values could cause unsafe changes

Correctness must come before speed.

16. CI/CD fundamentals

CI/CD turns code changes into controlled deployable artifacts.



The pipeline is fundamentally:

Commit
→ build
→ test
→ scan
→ package
→ publish
→ deploy
→ validate
→ promote

GitHub Actions, Jenkins, GitLab CI and Tekton implement different models and user experiences, but the engineering principles remain similar:

- deterministic builds
 - isolated jobs
 - immutable artifacts
 - automated testing
 - credentials separated from source code
 - repeatable environments
 - explicit promotion
 - auditable execution
 - fail fast
 - no manual production mutation outside controlled processes where practical
-

17. Software artifact management

Container registry

Stores container images.

Source Code
|
Build
|
network-worker:4.7.2
|
Registry

Kubernetes pulls images from the registry.

Artifact immutability

The same published version should not silently change.

Bad:

network-worker:4.7.2 today
!=
network-worker:4.7.2 tomorrow

Good:

4.7.2 always represents the same artifact

Versioning

Version artifacts so changes can be identified and rolled back.

Provenance

We should be able to answer:

Which commit produced this image?

Which pipeline built it?

Which dependencies were used?

Who approved promotion?

Which tests passed?

This is increasingly important for software supply-chain security.

Environment promotion

Prefer:

Build once

Artifact X

|

+--> Dev

+--> Test

+--> Staging

+--> Production

rather than rebuilding different binaries/images for every environment.

18. Deployment strategies

Rolling

Gradually replace old instances.

V1 V1 V1

↓

V2 V1 V1

↓

V2 V2 V1

↓

V2 V2 V2

Pros:

- efficient
- relatively simple

Risk:

- two versions temporarily coexist
-

Blue/green

Maintain:

Blue = current

Green = new

Test Green, then move traffic:

Users → Blue

becomes

Users → Green

Easy traffic rollback, but requires additional capacity.

Canary

Expose a small percentage of traffic to the new version.

95% → V1

5% → V2

Observe.

Then perhaps:

75/25

50/50

0/100

Progressive delivery

Broader concept:

Deploy gradually

+

measure health

+

automatically pause/continue/rollback

Signals might include:

- errors
- latency
- failed workflows
- business metrics

Application deployment vs network rollout

These must not be confused.

Application deployment

```
network-worker:v7
  |
Kubernetes
  |
Worker Pods
```

We are updating **software**.

Network configuration rollout

```
Policy/configuration change
  |
Network automation
  |
Routers/Switches
```

We are changing **network infrastructure**.

Both may use canaries, but the semantics differ.

Network rollout might be:

```
1 lab device
  |
2 pilot devices
  |
one site
  |
10% sites
  |
regional rollout
  |
global rollout
```

Validation might include:

BGP adjacency
reachability
route counts
packet loss
telemetry health

And rollback can be harder than simply switching application traffic back to an old Pod.

19. Testing architecture

Different test types prove different things.

Test	Main purpose
Unit	Small logic behaves correctly
Integration	Multiple real components interact
Component	One service behaves as a complete component
Contract	Interfaces agree
End-to-end	Whole user/workflow path succeeds
Infrastructure	Infrastructure definitions/configuration work
Failure	System tolerates failures
Device simulator/mocks	Network interaction logic can be exercised safely

Unit

Example:

```
Input device configuration
      |
Diff algorithm
      |
Expected change set
```

Fast and isolated.

Integration

Example:

```
Worker
  |
real database
  |
real queue
```

Tests interfaces between components.

Component

Run the network-worker service as a complete application while substituting external network devices.

Tests:

API contract
internal business logic
database interaction
job lifecycle

Contract testing

Tests producer/consumer assumptions.

Example:

Workflow service sends:

```
{  
  "device_id": "...",  
  "operation": "..."  
}
```

Worker must understand exactly this schema.

Useful in independently deployed microservices.

End-to-end

Example:

User submits change

```
      |  
API   |  
      |  
Workflow  
      |  
Queue |  
      |  
Worker |  
      |  
Simulated device  
      |  
Validation  
      |  
Audit result
```

This proves the overall path but is slower and harder to diagnose.

Infrastructure testing

Validate:

- Kubernetes manifests
 - IaC
 - permissions
 - routing
 - policies
 - environment assumptions
-

Failure testing

Deliberately simulate:

worker crash
queue unavailable
database timeout
device unreachable
API rate limit
pod termination
AZ failure

Production reliability depends heavily on these behaviours.

Device simulators/mocks

Never require hundreds of production routers to test ordinary code changes.

Use:

mocks
protocol simulators
virtual devices
lab networks
recorded responses

while still retaining selected real-device integration testing.

20. TDD as an engineering discipline

TDD is more useful as a design mechanism than as a slogan.

Basic loop:

```
Define behaviour
  |
Write small test
  |
Implement minimum behaviour
  |
Refactor
  |
Repeat
```

The important architectural value is:

Behaviour specification

You clarify:

| Exactly what should this component guarantee?

Small design feedback loops

Difficult-to-test code frequently reveals:

- excessive coupling
- hidden dependencies
- unclear responsibilities
- global state

Regression protection

Once a bug is understood:

```
Bug
  |
Test reproduces bug
  |
Fix
  |
Test prevents return
```

TDD does **not** mean every trivial line requires a test before it can exist.

The deeper principle is designing around explicit, testable behaviour.

21. Infrastructure as Code

IaC means infrastructure configuration is managed as versioned definitions rather than manual console operations.

It may define:

Networks
Subnets
Routing
Kubernetes clusters
IAM permissions
Databases
Object storage
Load balancers
DNS
Monitoring infrastructure

Architecture:

Git
|
Infrastructure Definition
|
Plan/Validation
|
Controlled Apply
|
Cloud/Private Infrastructure

Benefits:

Repeatability
Reviewability
Versioning
Automation
Auditability
Environment consistency
Disaster recovery

The important principle is very similar to Kubernetes:

Desired infrastructure state
|
v
IaC engine
|
v
Actual infrastructure

Again, declarative reconciliation appears.

22. Public, private, hybrid and multi-cloud

On-premises

Infrastructure physically operated within an organisation's own facilities or colocated environments under its direct operational model.

Private cloud

Cloud-like infrastructure dedicated to one organisation.

It may provide:

- self-service provisioning
- APIs
- automation
- elastic-like resource allocation

but remain privately controlled.

Public cloud

Shared cloud-provider infrastructure consumed as services.

Examples of service categories:

compute
networking
storage
managed databases
Kubernetes
identity
monitoring

Hybrid cloud

Combination of private/on-premises environments and public cloud.

On-Prem / Private
|
Secure Connectivity
|
Public Cloud

Multi-cloud

Using more than one cloud provider.

Cloud A
Cloud B
Cloud C

Multi-cloud does not automatically mean every application runs identically in every cloud.

Why banks often use hybrid architecture

Typical drivers include:

- existing mainframe/core systems
- regulatory requirements
- data-location controls
- security policies
- specialised hardware
- latency to internal systems
- phased migration
- operational control
- disaster-recovery strategy

Example:

Core banking systems
|
Private infrastructure
|
Secure connectivity
|
Public cloud analytics / application platform

Hybrid architecture increases flexibility but also increases networking, identity, governance and operational complexity.

23. Cloud conceptual mapping

Do not memorise three separate clouds first.

Learn the architecture concept.

Concept	AWS	Azure	GCP
Virtual network	VPC	Virtual Network	VPC
Subnet	Subnet	Subnet	Subnet
Compute VM	EC2	Virtual Machines	Compute Engine
Managed Kubernetes	EKS	AKS	GKE
Identity/access	IAM	Entra ID / Azure RBAC concepts	Cloud IAM
Object storage	S3	Blob Storage	Cloud Storage
Load balancing	Elastic Load Balancing	Azure LB/Application Gateway family	Cloud Load Balancing
Monitoring	CloudWatch	Azure Monitor	Cloud Monitoring

The portable architecture is:

Identity
|
Virtual Network
|
Subnets
|
Load Balancer
|
Kubernetes/Compute
|
Database + Object Storage
|
Monitoring

A strong platform architect understands this model before vendor terminology.

24. Performance engineering

Performance problems usually have a limiting resource or dependency.

Latency

Time required for one operation.

Example:

Device API request:
450 ms

Throughput

Amount of work completed per unit time.

Example:

20,000 device checks/minute

Low latency and high throughput are related but not identical.

CPU-bound

Worker spends most time computing.

Example:

processing massive configuration diffs
complex compression
heavy cryptographic processing

Scaling CPU or parallel computation may help.

I/O-bound

Worker spends most time waiting.

Typical network automation:

```
Worker
|
send API request
|
wait for router
|
receive response
```

This is heavily I/O-bound.

Concurrency can improve throughput.

Bottleneck

The slowest constrained part limits the system.

For example:

```
1000 workers
    |
    v
Database limited to 200 connections
```

Adding workers makes things worse.

The database is the bottleneck.

Profiling

Measure before optimising.

Examine:

```
CPU
memory
database time
network time
external API latency
lock contention
queue waiting time
```

Connection pooling

Reuse:

HTTP connections
database connections
device connections where safe

instead of constantly establishing expensive connections.

Caching

Avoid repeated expensive lookups where stale data is acceptable.

Batching

Instead of:

1000 API calls for 1000 records

use:

10 calls × 100 records

where the downstream interface permits it.

Pagination

Process large inventories incrementally.

100 devices
100 devices
100 devices
...

rather than loading millions simultaneously.

Lazy loading

Retrieve expensive information only when needed.

Example:

Device list:
id, name, status

Detailed configuration:
load only when user opens device

Rate limiting

Protect services and devices.

```
Workers
  |
Rate limiter
  |
Router API
```

Without it, automation can accidentally overload the equipment being automated.

Backpressure

If downstream components cannot keep up, upstream components must slow down.

Bad:

```
API generates 100k jobs/sec
Queue grows forever
Workers handle 1k/sec
```

Better:

```
Queue depth rises
  |
Backpressure
  |
Throttle producers
or reject/defer low-priority work
```

Backpressure is a fundamental distributed-systems concept.

25. Data engineering fundamentals

Network automation produces significant operational data.

Examples:

```
device inventory
interface counters
routing information
logs
configuration state
events
workflow history
alarms
telemetry
```

ETL

Extract
|
Transform
|
Load

Transform before loading into the analytical destination.

ELT

Extract
|
Load
|
Transform

Raw data is stored first and transformed later.

Modern analytical architectures frequently use combinations of both.

Batch

Process accumulated data periodically.

Example:

Every hour:
process telemetry files

Streaming

Process events continuously.

Router event
|
Stream
|
Processor
|
Seconds later
|
Dashboard/alert

Data lake

Stores large amounts of raw/semi-structured data.

Examples:

telemetry files
logs
configuration snapshots
raw events

Data warehouse

Optimised for analytical queries.

Examples:

network utilisation trends
capacity analysis
failure frequency
change-success trends
regional availability

Redshift, BigQuery and Snowflake are examples of analytical warehouse/platform concepts, but the important architecture is independent of the vendor.

Operational vs analytical database

Operational database:

"Give me workflow 7245."
"Update device status."
"Create approval record."

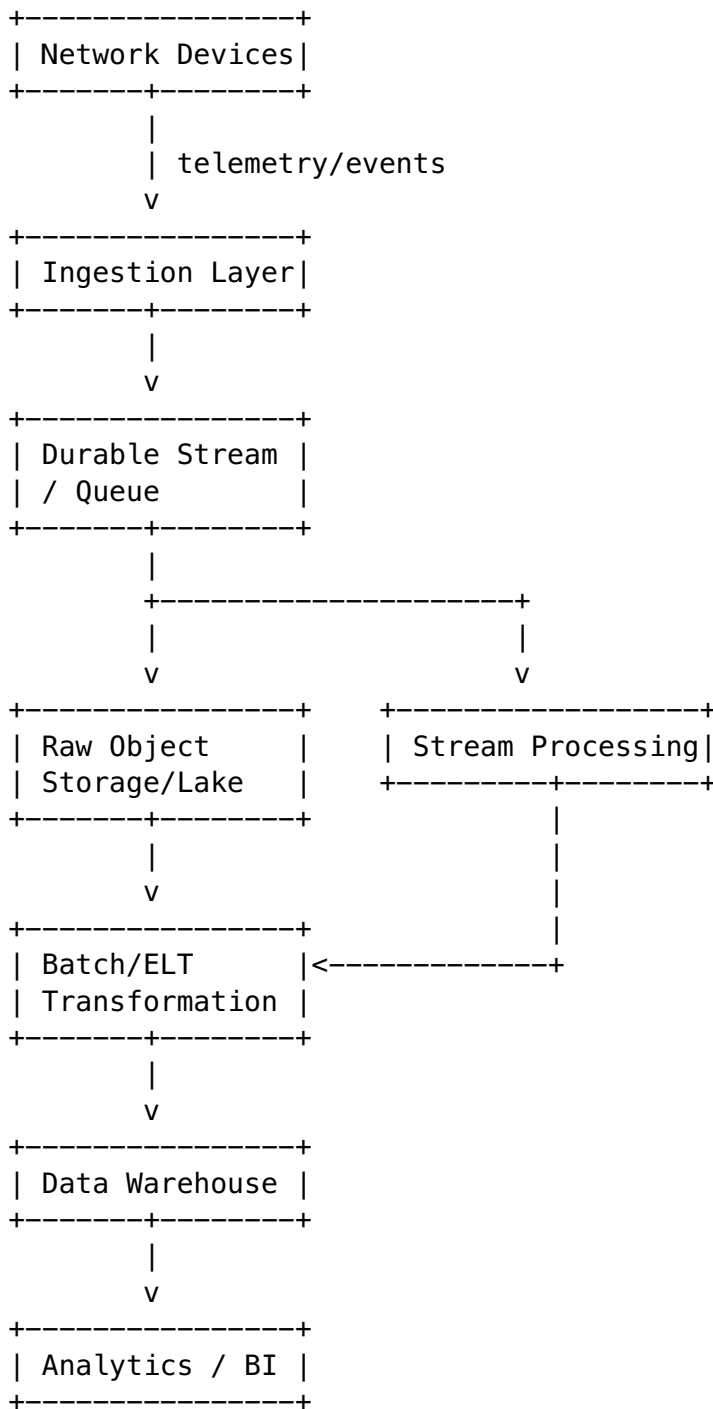
Optimised around live application transactions.

Analytical database:

"Calculate monthly failure rate
across 40 million changes grouped
by region and device family."

Different workloads justify different storage architectures.

Network data pipeline



Raw storage is important because transformed data can be regenerated if transformation logic changes.

26. Schema evolution and data quality

Distributed systems evolve.

Suppose V1 telemetry is:

```
{  
  "device": "r1",  
  "cpu": 48  
}
```

Later V2 adds:

```
region  
device_type  
timestamp precision
```

Consumers cannot all upgrade simultaneously.

Backward compatibility

A newer schema should ideally avoid unexpectedly breaking older consumers.

Safe evolution often favours:

adding optional fields

over:

renaming/removing required fields immediately

Schema versioning may become necessary.

Validation

Validate data at boundaries.

Example:

CPU = -900%

should not silently enter analytical systems.

Check:

- type
 - range
 - required fields
 - identifiers
 - timestamps
 - schema versions
-

Missing values

Missing value does not always mean zero.

Example:

```
interface utilisation = 0
```

means something different from:

```
interface utilisation = unknown
```

Preserve that semantic distinction.

Duplicate events

Distributed systems commonly deliver events more than once.

For example:

```
event_id=123  
event_id=123
```

Consumers should often support idempotent processing/deduplication.

Data-quality monitoring

Monitor metrics such as:

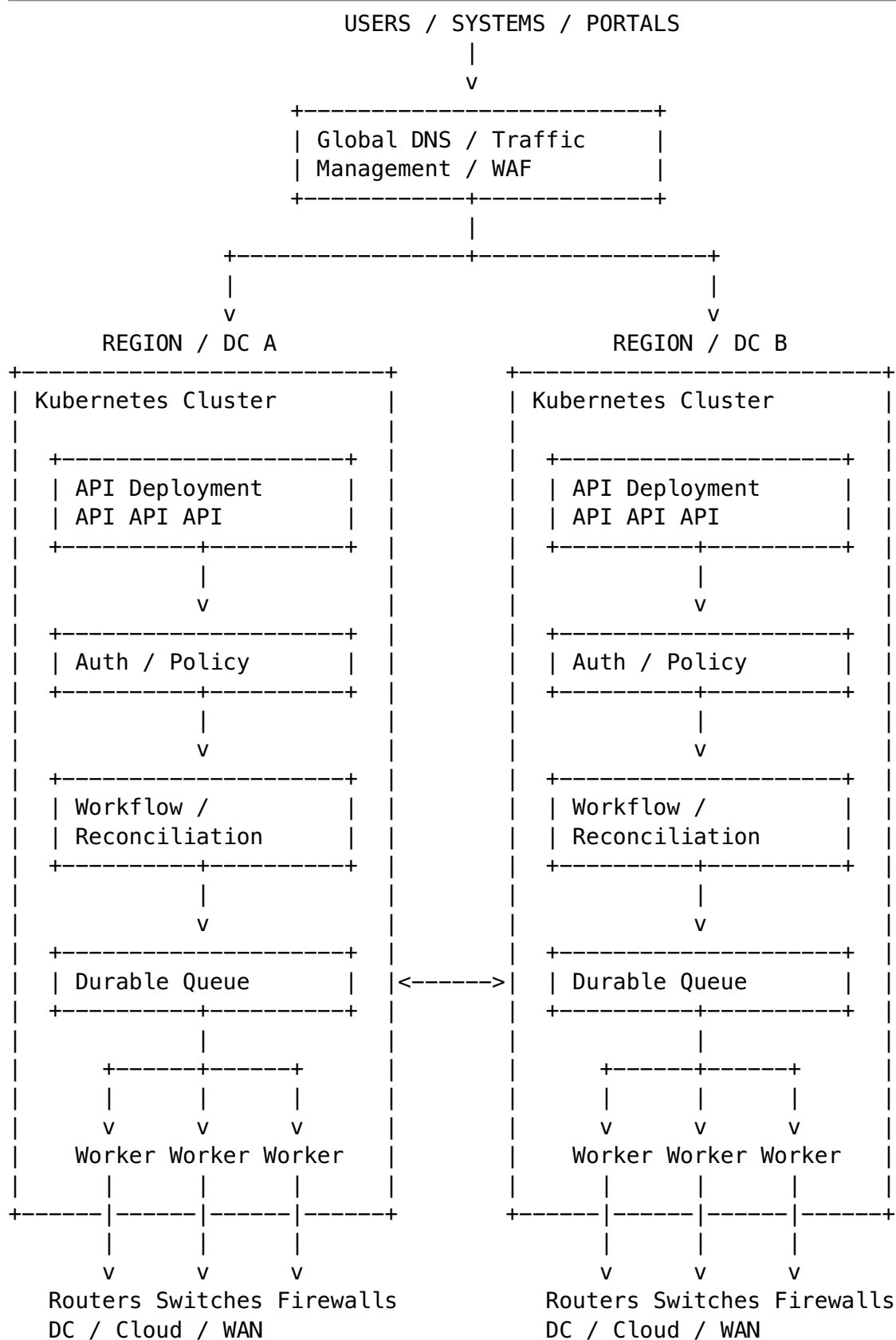
```
event volume  
missing-field percentage  
duplicate percentage  
invalid-record percentage  
processing lag  
late events  
unexpected schema versions
```

Otherwise a pipeline may appear technically "healthy" while producing incorrect analytics.

Putting everything together

Now combine the entire Day 4 architecture with Day 3.

Global highly available network-automation platform

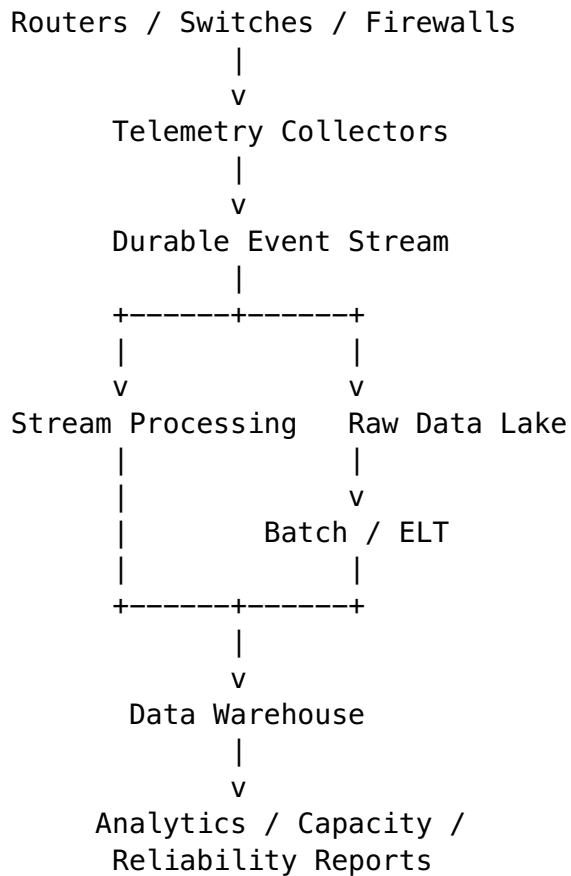


SHARED / REPLICATED PLATFORM SERVICES

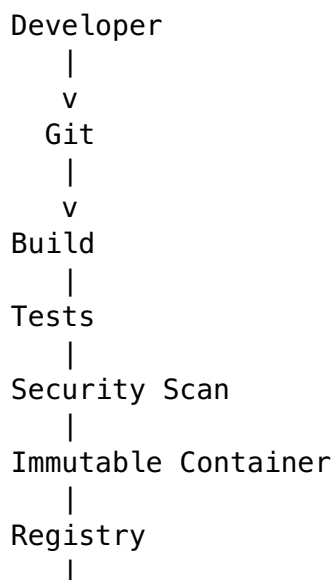
Source of Truth / Inventory / IPAM
Relational Database
Audit Store

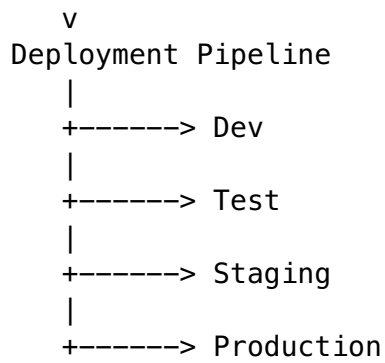
Cache	
Object Storage	
Search / Log Index	
Secret Manager	
IAM	

TELEMETRY DATA PLANE

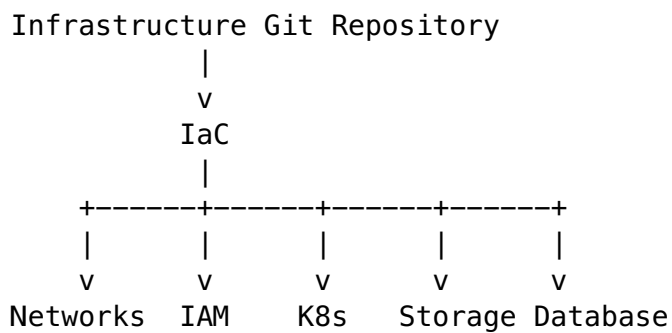


DELIVERY PLATFORM



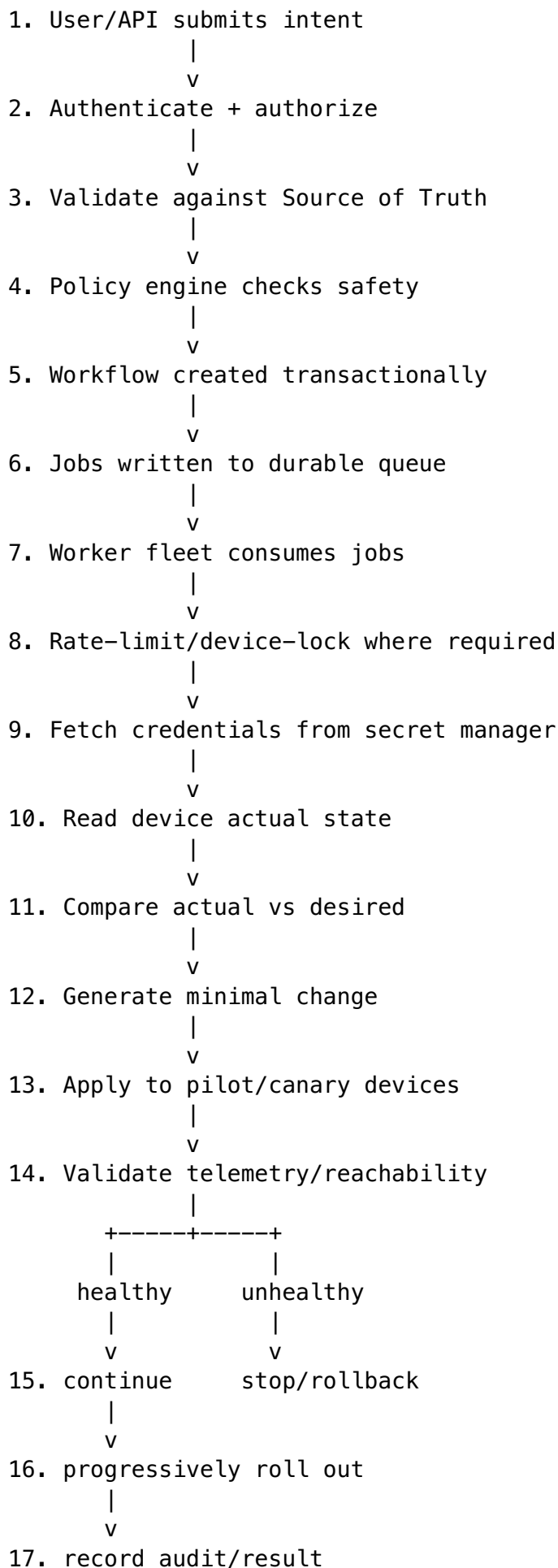


INFRASTRUCTURE MANAGEMENT



How one network change flows through this architecture

Suppose someone wants to change a routing policy on 5,000 devices.



|
v

18. reconciliation continues

Notice how many Day 1–4 concepts participate in just one operation:

IP / routing / TCP	Day 1–2
BGP / VRF / network design	Day 2
API / inventory / SoT / workflows	Day 3
Queues / workers / reconciliation	Day 3
Kubernetes	Day 4
Distributed systems	Day 4
Databases/cache	Day 4
CI/CD/IaC	Day 4
Cloud/HA	Day 4
Telemetry/data engineering	Day 4

The most important architectural principle from Day 4

A large infrastructure platform should avoid depending on any individual machine, Pod or process.

Instead:

Compute is replaceable.
State is durable.
Work is recoverable.
Changes are versioned.
Deployments are repeatable.
Failures are expected.
Reconciliation restores desired state.
Observability tells us whether reality matches our expectations.

That combination—**declarative desired state + durable distributed state + replaceable compute + controlled automation + continuous reconciliation**—is the foundation of modern cloud and network-automation platforms.